ANLP Assignment 1

Yash More, 2024114004

1 Introduction and Cipher Analysis

The task is character-level decryption framed as sequence-to-sequence translation: given a ciphertext line X, produce the plaintext line Y. Before choosing an architecture I looked at the data. Inspecting `brown_plain.txt` and `brown_cipher.txt` showed three things. Every plaintext character maps to exactly 8 bits, so the encoding is fixed-length and the two files align character by character. Newlines are left unencrypted, which is why I process the data line by line. Most importantly, the same 8-bit block decodes to different characters in different places, which rules out a static substitution table. A model therefore cannot succeed by memorising a lookup table; it has to condition on where in the line a block sits. The cipher is position-dependent, so I expected the way a model represents position to matter more than its attention or normalization scheme. That is the hypothesis the ablation tests, and it is broadly what the results show.

Windowing. I also assumed a line can be cut into independent 64-character windows, each its own training example, on the basis that a window carries enough context to be decrypted alone. This is not obviously safe, since it discards dependencies crossing a boundary, so I tested it: Section 5.1 compares both rounds and windowing improves every configuration by more than an order of magnitude. Window boundaries are multiples of 64, itself a multiple of the 8-bit block size, so a window never starts mid-character.

2 Experimental Setup

Data and hardware. The dataset has 5,000 aligned (ciphertext, plaintext) lines, split 80/10/10 by a seeded shuffle. Splitting happens at the line level before windowing, so no window of a test line appears in training. Windowing gives 39,683 train, 4,685 validation and 4,782 test windows, identical for all five configurations. Each configuration trained on its own RTX 2080 Ti (11 GB) on the Ada cluster, four in parallel, with FP16 autocast.

Tokenization. Both tokenizers are Byte-Pair Encoding written from scratch, with no library used for training, encoding, decoding or serialization. The plaintext side learns merges over characters with whitespace pre-tokenization (vocabulary 5,000), keeping each space attached to the following word so concatenating tokens restores the string exactly. On the cipher side what BPE runs *over* matters: I first group the bit string into 8-bit blocks, map each to one symbol of a 256-symbol alphabet, then learn merges over that symbol stream (vocabulary 4,096). Running BPE on the raw `0/1` string instead, which I tried first, lets merges straddle 8-bit boundaries and destroys the byte alignment the cipher depends on. The incremental formulation, rather than rescanning per merge, brings cipher vocabulary training down to 21 seconds. Both tokenizers train on the training split only and round-trip losslessly. Tokenization (BPE over the byte-symbol stream) and windowing (training-example segmentation) are separate things that both involve the number 64.

Batching. I pad each batch only to its own longest member and group similar lengths together, reshuffling pools every epoch. Batches are packed to a budget of 4,096 padded positions rather than a fixed sequence count, since attention memory grows with $B \cdot L^2$ and length grouping puts the longest sequences in one batch, so sizing by the mean overflows on the worst one. This matters because I ran out of memory on the first full-line attempt: 16 sequences of 1,385 tokens need about 491 MB for a single encoder attention matrix in FP16, and with four encoder layers plus decoder self- and cross-attention all retained for the backward pass, that one batch exhausts

an 11 GB card even though the average line is only 296 tokens. Sizing batches by the mean is the wrong statistic when the peak is what allocates. The budget dropped worst-case encoder attention from 491 MB to 17 MB per layer, a factor of 29, and gives 648 optimizer steps per epoch for every configuration.

Hyperparameter	Value	Notes
`d_model` / `d_ff`	256 / 1024	4× expansion
`n_heads`	8	$d_k = 32$; $n_{\text{kv_heads}} = 2$ for C3
`num_layers`	4 enc + 4 dec	Identical in all five configurations
`plain / cipher BPE vocab`	5000 / 4096	Both from scratch
`learning_rate`	7×10^{-4}	Linear warmup (10%) + cosine decay
`dropout` / `wd` / `ls`	0.3 / 0.05 / 0.2	Weight decay, label smoothing
`max_tokens`	4096 / batch	Token budget, not a fixed count; 648 steps/epoch
`epochs`	70	≈45,000 optimizer steps
C5 only, in place of the two BPE vocabularies		
`byte vocab`	259	256 raw bytes plus 3 special tokens, both sides
`patch_size` / `n_local_layers`	8 / 1	Fixed-stride patching; one local byte encoder and decoder

Table 1: Hyperparameters. All five configurations share depth, width, head count, learning rate, schedule and optimizer steps per epoch, and differ only in the single ablated component. C5 replaces the two BPE vocabularies with a 259-token byte vocabulary and wraps the same 4 global blocks in one local byte encoder and decoder, so the comparison is depth-matched.

Budget and evaluation. All five ran on one schedule and were stopped for compute time rather than at convergence. C1 to C4 are reported at epoch 70, where they were still improving slowly; C5 is reported at its best checkpoint, epoch 29, after which a further 12 epochs gave no improvement, so it had plateaued. All metrics are computed on decoded text rather than token IDs, which is what makes the five comparable: each line's windows are decoded back to characters and concatenated in order, reassembling 4,782 test windows into exactly the 500 lines every model is scored on, verified to reconstruct the references with zero mismatches. Character accuracy, exact match and Levenshtein distance are character-level; BLEU and ROUGE are word-level. Scoring stitched text also removes an earlier mismatch where C1 to C4 emitted BPE tokens and C5 emitted bytes, so n-gram overlap is now meaningful for all five.

3 Hyperparameter Search

The configuration in Table 1 came out of the exploratory runs in Figure 1. Panel (a) suggested the task needed a large model, but the real problem was upstream: BPE over the raw bit string was destroying byte alignment, and once the cipher representation was fixed the apparent capacity wall mostly disappeared, which is why the final models use `d_model=256`. Panel (b) shows shorter sequence lengths giving worse loss, which I traced to the learning rate rather than to capacity, since those runs used 1×10^{-4} and the scheduler decayed it further. Panel (c) covers the higher rates I tried afterwards, and the move away from running BPE on the cipher.

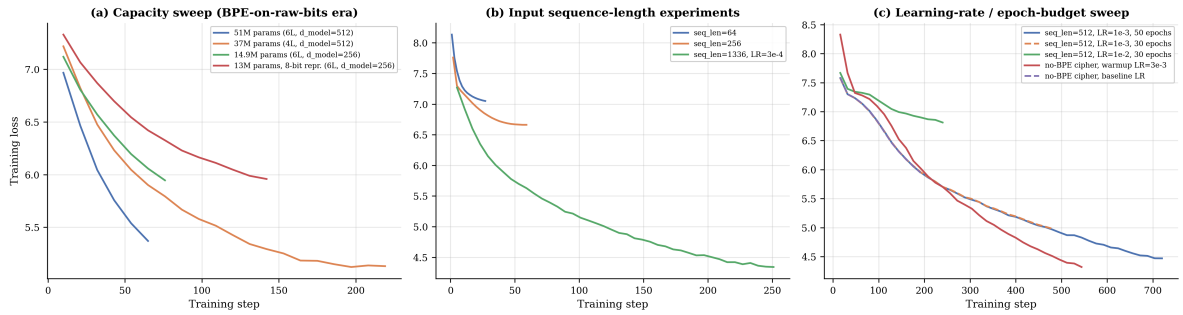


Figure 1: Exploratory runs, training loss vs. step. (a) capacity sweep during the BPE-on-raw-bits era, (b) input sequence-length experiments, (c) learning-rate and epoch-budget sweep plus the move away from BPE on the cipher.

4 Model Configurations

Config	Change from C1	Description
`C1`	Baseline	Sinusoidal absolute PE, multi-head attention, Pre-LayerNorm; from-scratch BPE on both sides
`C2`	RoPE [2]	Sinusoidal PE replaced by rotary embeddings inside attention
`C3`	GQA [3]	Grouped-Query Attention, $n_{\text{kv_heads}} = 2$ instead of 8
`C4`	RMSNorm [4]	LayerNorm replaced by RMSNorm
`C5`	BLT-style [5]	No BPE on either side; cipher read as a raw bit stream patched into bytes by a local encoder, plaintext written as raw UTF-8 bytes

Table 2: Architectural change in each configuration relative to C1. The changes are independent, not cumulative: C3 and C4 both keep C1's sinusoidal encoding, so C3 is not C2 plus GQA. That is what lets each row of Table 3 be attributed to a single component.

5 Results

Metric	C1 (Base)	C2 (RoPE)	C3 (GQA)	C4 (RMSNorm)	C5 (BLT)
Parameters	10.99M	10.99M	9.80M	10.98M	**5.53M**
val_loss	0.3034	0.1347	0.3073	0.2850	**0.0745**
Char Accuracy	29.12%	52.13%	28.30%	29.79%	**99.96%**
Seq Accuracy	2.20%	25.00%	1.80%	1.80%	**83.40%**
Levenshtein	18.54	6.57	18.60	17.18	**0.23**
BLEU	0.7766	0.9456	0.7727	0.7859	**0.9951**
ROUGE-1	0.9018	0.9768	0.9002	0.9067	**0.9982**
ROUGE-2	0.8200	0.9571	0.8160	0.8274	**0.9971**
ROUGE-L	0.9015	0.9768	0.9000	0.9065	**0.9982**

Table 3: Test metrics on all 500 held-out lines, greedy decoding, scored on decoded text. Bold marks the best in each row, underline the best among C1-C4. `val_loss` is comparable within C1-C4 only, since C5 predicts over a 259-token byte vocabulary.

Reading character accuracy carefully. Character accuracy is position-wise, comparing pred$[i]$ to ref$[i]$, so one inserted or deleted character shifts everything after it and collapses the score even when the text is nearly right. C2 is exactly that case: 52.13% accuracy alongside a mean edit distance of only 6.6 on lines averaging 581 characters, barely 1% of the characters. This is genuine alignment drift and not truncated decoding, since mean prediction-to-reference length ratios are 0.996, 1.004, 0.996, 0.996 and 1.000 for C1 to C5. Edit distance and BLEU are the metrics to trust here.

C2 (RoPE) is the only C1-C4 change that improves accuracy. It cuts edit distance from 18.54 to 6.57, a factor of 2.8, and lifts exact-line match from 2.20% to 25.00%, a factor of 11. This fits the position-dependent picture from Section 1: RoPE injects position into the attention dot

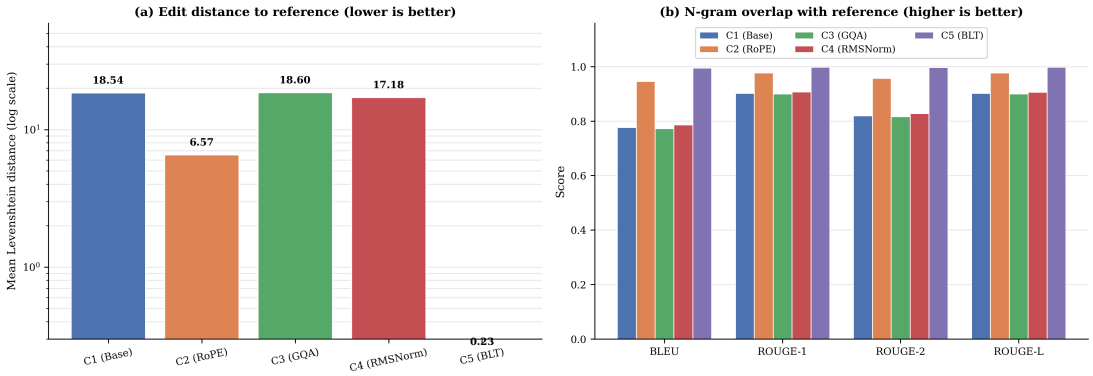


Figure 2: Edit distance (log scale) and word-level n-gram overlap. These capture different failure modes: edit distance is sensitive to how many characters are wrong, while BLEU and ROUGE reflect whether the output is the right words in the right order.

product at every layer, so the positional signal stays sharp throughout the network, whereas sinusoidal encoding adds it to the input embedding once and lets it dilute through four layers of mixing.

C3 (GQA) and C4 (RMSNorm) leave accuracy unchanged. Holding positional encoding fixed, both land beside the baseline on every metric: edit distances of 18.60 and 17.18 against C1's 18.54, and validation losses within 0.023 of each other, so I would not read the ordering among these three as meaningful. Neither helps nor hurts, which is what they were designed for: both target efficiency rather than representational power, and Section 6 shows they deliver exactly that.

C5 (BLT) solves the task, reconstructing 83.4% of held-out lines exactly and averaging 0.23 characters of error per line. Reading the ciphertext as raw bits removes the tokenizer as an intermediary, so position within the line is available directly rather than inferred through variable-length subword units. The four results share one explanation: C5 has position exactly, C2 keeps it sharp at every layer, and C1, C3 and C4 work from a signal injected once and then diluted. The ranking is about positional fidelity, not capacity.

5.1 Effect of Windowed Training

I first ran the whole ablation on full lines under otherwise identical settings, and it failed. Comparing the two rounds isolates what the windowing assumption is worth.

	C1	C2	C3	C4
Levenshtein, full-line	429.5	474.7	423.2	420.6
Levenshtein, windowed	**18.54**	**6.57**	**18.60**	**17.18**
BLEU, full-line	0.021	0.014	0.021	0.021
BLEU, windowed	**0.777**	**0.946**	**0.773**	**0.786**

Table 4: Full-line versus 64-character windowed training, same architectures, tokenizers, learning rate and token budget. The budgets differ: the full-line round ran 45 epochs and the windowed round 70. The gap is far too large to be a budget effect, since the full-line models had not begun to decrypt at all rather than being partway to a solution.

The full-line round is not just worse, it is uninformative: all four sit between 420 and 475 in edit distance with BLEU near 0.02, which is what a model that has learned plaintext letter statistics but cannot decrypt looks like, and the RoPE advantage is invisible. Had I stopped there I would have reported a null ablation. The gains also fit the positional account, since the configuration that gains most is C2.

REF	Isaac Amsel an year old boy and witness at the trial he is the son of Isaac Amsel senior former member of the search party killed earlier in a skirmish in So Paulo …
C5	Isaac Amsel an year old boy and witness at the trial he is the son of Isaac Amsel senior former member of the search party killed earlier in a skirmish in So Paulo …
C2	Isac Amsel an year old boy and witness at the trial he is the son of Isaac Amsel senior former member of the search party killed earlier in a skirmish in So Paulo …
C1	Isac Amosel an year old boy and witness at the trial he is the son of Isac Cossel senior former member of the search party killed earlier in a skirmish in So Paul …
C4	Isac Amosel an year old boy and witness at the trial he is the son of Isac Cowel senior former member of the search party killedd earlier in a simpmish in So Paul …

Table 5: Test line 152, chosen because each model's edit distance on it is close to its median. C5 is exact, C2 drops one character, and C1 and C4 slip on proper nouns and rare words. C3 closely resembles C1.

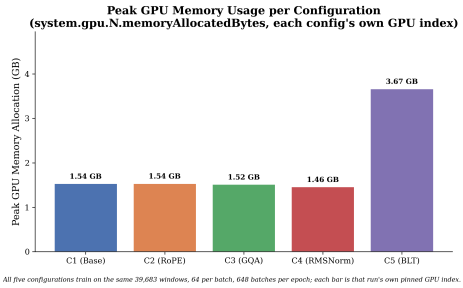
5.2 Qualitative Error Patterns

C1, C3 and C4 have largely learned the mapping but lose individual characters, and those losses concentrate on proper nouns and rare words such as *Amsel* and *Paulo*, where a language-model prior over English cannot fill the gap. That is the signature of a model still leaning partly on plaintext statistics instead of decrypting outright. C2 makes that error far less often, and C5 not at all here.

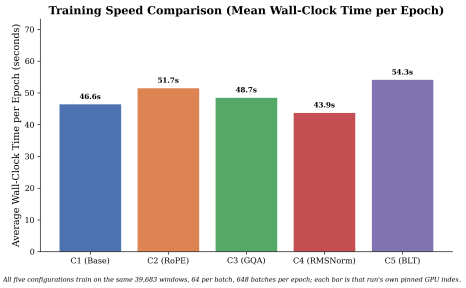
6 Efficiency and the BLT Tradeoff

Config	Parameters	Peak GPU memory	s/epoch	Levenshtein
C1 (Base)	10,986,888	1.537 GB	46.6	18.54
C2 (RoPE)	10,986,888	1.539 GB	51.7	6.57
C3 (GQA)	9,802,632	1.519 GB	48.7	18.60
C4 (RMSNorm)	10,981,512	**1.465 GB**	**43.9**	17.18
C5 (BLT)	**5,532,931**	3.668 GB	54.3	**0.23**

Table 6: All five train on the same 39,683 windows at 648 optimizer steps per epoch, so one epoch is one pass over identical data in every row and the columns are directly comparable. Memory is read from each run's own pinned GPU index.



(a) Peak GPU memory.



(b) Wall-clock time per epoch.

Figure 3: System metrics, each configuration on its own RTX 2080 Ti.

The memory and accuracy tradeoff for C5. This is where the token-free approach shows a real and two-sided cost. C5 is at once the most accurate and by far the smallest, using 5.53M parameters against C1's 10.99M, a 50% reduction that comes almost entirely from the vocabulary: predicting over 259 byte tokens instead of 5,000 subword tokens shrinks the embedding and output projection sharply. But C5 pays in activation memory, peaking at 3.67 GB against C1's 1.54 GB, roughly 2.4× more, and it is about 17% slower per epoch.

For C5, windowing is not merely a convenience but a hard architectural constraint: a full line costs about 4,800 bit tokens on average and 21,360 at the longest, and a single local-encoder attention matrix at 21,360 positions needs roughly 7.3 GB in FP16 before any backward pass, which does not fit an 11 GB card.

The reason for the memory cost is sequence length. C5 reads one token per bit, so a 64-character window becomes 514 encoder positions against about 33 BPE tokens for C1 to C4. Attention cost grows with the square of that length, and no parameter saving offsets a roughly 15× longer sequence. Parameters are stored once, whereas activations are held for every position in every layer for the backward pass, so the two move independently, and here in opposite directions. The effect is larger at inference: decoding the test set takes about 30 minutes for C5 against 39 seconds for C2. The honest summary is that BLT trades memory and time for parameters and accuracy rather than dominating on every axis.

Attributing memory correctly requires care. My first attempt at this comparison reported an implausible 19.77 GB for C1, more than an 11 GB card can hold, and two separate bugs were responsible. The field I read as "GPU memory in GB" (`system.gpu.N.memoryAllocated`) is actually a percentage of that GPU's total VRAM. More subtly, WandB polls *every* GPU on the machine rather than the one a job uses, so each of four concurrent runs logs all four GPUs' readings and taking "the first GPU-memory column found" silently returns a sibling run's memory. Correct attribution requires reading `system.gpu.N.memoryAllocatedBytes` at the specific index each configuration was pinned to, which is why the launch script fixes one configuration per GPU.

What the other two components buy. RoPE buys a factor of 2.8 in edit distance for an 11% speed cost, with no extra parameters or memory. GQA buys 10.8% fewer parameters and RMSNorm 6% speed and the lowest memory of all five, both with accuracy unchanged: they behave exactly as designed, as accuracy-neutral efficiency measures that do not address what makes this task hard.

7 Conclusion

The ablation shows that positional encoding is the component that matters on this task and that attention and normalization are not. This follows from the cipher being position-dependent: the configurations that preserve position best, C5 then C2, are the ones that actually decrypt. Matching a component to the structure of the problem mattered far more than adding generally strong components, and the largest single gain came from how training examples were framed rather than from architecture or tuning.

Three limitations are worth stating. C1 to C4 were stopped at a fixed 70-epoch budget while still improving, so their numbers are a lower bound and the gap to C5 would likely narrow with a longer schedule, though C2's advantage was stable throughout. The learning rate was shared rather than tuned per configuration. And the full-line round ran 45 epochs against the windowed round's 70, so that comparison is not budget-matched, though the size of the gap makes a budget explanation implausible.

Links

WandB Project: https://wandb.ai/yash-more1224-international-institute-of-information-tec/anlp-assignment-1

Reported runs: `run-C1-bpew-long`, `run-C2-bpew-long`, `run-C3-bpew-long`, `run-C4-bpew-long` and `run-C5-bpew-long` are the reported windowed runs; all curves and the Section 6 efficiency numbers come from these five. `run-C1-bpe` to `run-C4-bpe` are the superseded full-line round in Section 5.1. All other runs are exploratory.

Hugging Face Checkpoints: https://huggingface.co/yashmore1224/ANLP-A1

This repository holds all five reported checkpoints together with both from-scratch BPE tokenizers, which are required to use C1-C4: `checkpoint_C1_epoch70.pt`, `checkpoint_C2_epoch70.pt`, `checkpoint_C3_epoch70.pt`, `checkpoint_C4_epoch70.pt`, `checkpoint_C5_epoch29.pt`,

`plain_bpe.json`, `cipher_bpe.json`.

References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems 30 (NeurIPS 2017), pages 5998–6008.

[2] Su, J., Ahmed, M. H. M., Lu, Y., Pan, S., Bo, W., and Liu, Y. (2024). *RoFormer: Enhanced transformer with rotary position embedding*. Neurocomputing, 568:127063.

[3] Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. (2023). *GQA: Training generalized multi-query transformer models from multi-head checkpoints*. Proceedings of EMNLP 2023, pages 4895–4901.

[4] Zhang, B. and Sennrich, R. (2019). *Root mean square layer normalization*. Advances in Neural Information Processing Systems 32 (NeurIPS 2019), pages 12360–12371.

[5] Pagnoni, A., Pasunuru, R., Rodriguez, P., Nguyen, J., Muller, B., Li, M., Zhou, C., Yu, L., Weston, J., Zettlemoyer, L., Ghosh, G., Lewis, M., Holtzman, A., and Iyer, S. (2025). *Byte latent transformer: Patches scale better than tokens*. Proceedings of ACL 2025. arXiv:2412.09871.